

ExpenseEase — Mid-Semester Project Report

Course: Software Engineering & Project Management (CSN15101)

Section: E **Group Number:** [Your Group Number]

Instructor: Dr. Satarupa Chakrabarti

Submission Date: September 2026

Team Members

Name	Registration Number
Shreeya Srivastava	20243267
Shraddha Sharma	20243266
Rudransh Pratap Singh	20243243
Sachit Jain	20243245

1. Problem Statement and Motivation

Managing shared expenses within a group — whether it is a college trip, shared accommodation, or a project team dinner — is inherently messy. The typical approach is a WhatsApp thread of receipts and informal "you owe me" messages, which quickly becomes untrackable. People forget who paid what, calculations are done mentally and are prone to error, and settling debts often requires multiple rounds of back-and-forth negotiation.

Existing solutions like Splitwise exist but are either locked behind paywalls for core features, require app installation, or are overkill for a student group that simply wants to know "who owes whom and how much." There is also a trust problem: when an app calculates balances, users often cannot verify how those numbers were arrived at.

This project addresses this problem directly. ExpenseEase is a web-based expense-splitting application built specifically for small, trust-based groups such as student flat-mates, trip groups, or project teams. It records shared expenses, computes individual balances transparently using a published formula, and minimises the number of cash transfers needed to clear all debts. Every calculation is deterministic — the same input always produces the same output — so any member can manually verify the results.

The motivation for building this system came from a real pain point experienced by the team itself during inter-college events and shared accommodation. The project is also an

opportunity to put software engineering principles into practice in a meaningful, self-contained domain.

2. Project Objectives

The primary goal of ExpenseEase is to provide a group with a single, honest ledger for shared expenses. Specifically, the system is designed to:

1. **Eliminate manual calculation errors** — all split arithmetic is performed server-side and validated by automated tests.
 2. **Support multiple split strategies** — equal division, exact amounts, and percentage-based splits cover the most common real-world scenarios.
 3. **Produce verifiable balances** — the balance formula is documented and the invariant (all balances in a group sum to zero) is enforced in code.
 4. **Minimise settlement transactions** — a greedy debt-simplification algorithm reduces the number of transfers needed to clear all debts.
 5. **Maintain a full audit trail** — every expense and settlement is logged so members can trace how the current balance was reached.
 6. **Be accessible without installation** — the application runs in any modern browser; no mobile app is required.
 7. **Protect member data** — passwords are hashed with bcrypt, every API endpoint requires a valid JWT, and group data is scoped to members only.
-

3. Scope of the Project

3.1 Features Included

Feature	Description
User registration and login	Email + password authentication with JWT session tokens (7-day expiry)
Group creation	Any authenticated user can create a group and becomes its administrator
Member management	Group admin can add registered users by email address
Expense recording	Members record expenses with description, amount, category, date, payer, and split type

Feature	Description
Equal split	Amount divided equally; remainder paise distributed deterministically by user ID
Exact split	Each participant's share entered manually; validated to sum to total
Percentage split	Each participant's percentage entered; validated to total 100; shares computed server-side
Balance calculation	Net balance per member computed as total paid minus total owed, adjusted for settlements
Debt simplification	Greedy algorithm reduces the number of cash transfers needed to clear all debts
Settlement recording	Any member can record a payment between two group members
Activity log	Every expense and settlement action is timestamped and logged per group
Cross-group dashboard	Logged-in user sees total "you owe" and "owed to you" across all groups
Expense editing	Expense creator or group admin can edit any field, including split details
Expense deletion	Expense creator or group admin can delete an expense

3.2 Features Excluded

The following were considered but deliberately excluded from the mid-semester deliverable:

- **Password reset** — no email delivery mechanism is in scope; a placeholder message is shown.
- **Notifications** — no email or push notification system.
- **File/receipt attachments** — expenses are text-only.
- **Currency support beyond INR** — all amounts are stored in paise (Indian Rupees only).
- **Recurring expenses** — no auto-repeat functionality.

- **Expense categories as a dropdown** — category is a free-text field; no enforcement of a fixed taxonomy.
- **Group deletion or archiving** — groups persist indefinitely.
- **Mobile application** — the system is web-only; no native Android or iOS app.
- **Offline mode** — no service worker or local-first data strategy.

3.3 Assumptions and Constraints

Assumptions:

- All group members must be registered users before they can be added to a group; adding by name alone is not supported.
- All monetary amounts are in Indian Rupees. Amounts are entered in rupees (e.g., ₹ 150.00) and stored as integers in paise (15000) to avoid floating-point rounding.
- Percentage splits accept whole-number percentages only (e.g., 33% is valid; 33.3% is not).
- The group administrator who creates the group is the only user who can add new members.
- Settlements do not automatically close or archive; balances accumulate across all expenses and settlements.

Constraints:

- The system requires Node.js 20+ and a PostgreSQL 16 database.
- The Docker Compose deployment is the supported production setup; local development requires manual environment variable configuration.
- There is a rate limit of 20 login or registration attempts per minute per IP to prevent brute-force attacks.
- JWT tokens expire after 7 days; there is no token refresh mechanism in the current version.

4. Stakeholders

Stakeholder	Interest	Influence
Group members	Accurate shared costs	High
Group admins	Membership and data oversight	High
Course evaluator	Evidence of engineering practice	High

Stakeholder	Interest	Influence
Development team	Deliverable quality and learning	High
Future maintainers	Clear design and tests	Medium

5. Requirements Specification

Functional Requirements

- User registration and login.
- Group creation and member addition.
- Expense CRUD operations.
- Split validation (equal, exact, percentage).
- Balance calculation and debt simplification.
- Settlement recording.
- Activity log and dashboard summaries.

Non-Functional Requirements

- bcrypt password hashing.
- JWT-based authentication.
- Validated input.
- Dockerized PostgreSQL.
- Balance invariants.

6. System Design

The system is a browser-based SPA (Single Page Application) that communicates with an Express REST API. The backend uses Prisma for database access, and balances are computed dynamically from expenses and settlements to prevent stale data.

7. Technology Stack

Layer	Choice	Justification
Frontend	React/Vite/Axios	Fast SPA and API integration.

Layer	Choice	Justification
Backend	Node.js/Express	Small REST service with clear routes.
Validation	Zod	Declarative input contracts.
Database	PostgreSQL/Prisma	Dockerized relational persistence.
Security	bcrypt/JWT/helmet	Password hashing and protected routes.
Tests	Vitest/Supertest	Fast unit and API-oriented testing.

8. Implementation Progress

The backend of ExpenseEase is built using **Node.js** and **Express**, with a focus on modularity and scalability. Key backend features include:

1. Authentication and Authorization:

- User registration and login are implemented with bcrypt for password hashing and JWT for session management.
- Role-based access control ensures that only group admins can add or remove members.

2. Expense Management:

- CRUD operations for expenses allow users to create, read, update, and delete expenses.
- Split validation ensures that all expense shares (equal, exact, or percentage) are accurate and consistent.

3. Balance Calculation:

- Balances are dynamically calculated from expenses and settlements, ensuring data consistency.
- Debt simplification uses a greedy algorithm to minimize the number of transactions required to settle debts.

4. Database Integration:

- PostgreSQL is used as the relational database, with Prisma as the ORM for schema management and queries.
- Dockerized PostgreSQL ensures consistent development and production environments.

5. Testing:

- The backend is thoroughly tested with **Vitest** and **Supertest**, covering unit tests, integration tests, and API tests.
- Test cases include edge scenarios such as invalid splits, unauthorized access, and concurrent updates.

8.2 Frontend Implementation

The frontend is a **React**-based Single Page Application (SPA) built with **Vite** for fast development and optimized builds. Key frontend features include:

1. User Interface:

- A responsive and intuitive UI allows users to register, log in, create groups, add members, and manage expenses.
- The dashboard provides a summary of balances, activity logs, and group details.

2. Expense Workflow:

- Users can add expenses with descriptions, amounts, categories, and split types.
- Real-time validation ensures that inputs are correct before submission.

3. Dashboard and Activity Log:

- The dashboard displays "you owe" and "owed to you" balances across all groups.
- The activity log provides a detailed history of all expenses and settlements.

4. API Integration:

- Axios is used for seamless communication with the backend API.
- Error handling and loading states are implemented to enhance the user experience.

8.3 Current Status

- **Backend:** Fully functional with all core features implemented and tested.
 - **Frontend:** Supports the complete registration-to-expense workflow, including group creation, member management, and expense recording.
 - **Testing:** 179 tests (157 backend, 22 frontend) have been successfully executed, ensuring high code quality.
-

9. Testing Performed

9.1 Test Strategy

The testing strategy for ExpenseEase includes the following levels:

1. Unit Testing:

- Focuses on individual functions and modules, such as split validation and balance calculation.
- Ensures that each component behaves as expected in isolation.

2. Integration Testing:

- Verifies the interaction between different modules, such as the API endpoints and the database.
- Ensures that data flows correctly through the system.

3. End-to-End Testing:

- Simulates real user workflows, such as registering, creating a group, adding members, and recording expenses.
- Ensures that the system works as a whole.

9.2 Test Results

On **30 September 2026**, the following tests were executed:

• Backend Tests:

- 157 tests covering authentication, group management, expense CRUD, split validation, balance calculation, and debt simplification.
- All tests passed successfully.

• Frontend Tests:

- 22 tests covering UI components, form validation, and API integration.
- All tests passed successfully.

9.3 Evidence

- The `npm test` command output confirms that all tests passed without errors.
 - Linting (`npm run lint`) and build (`npm run build`) were also successful.
-

10. Project Management Status

10.1 Development Approach

The project follows an **iterative and incremental SDLC**. Each increment delivers a working subset of features, which is then tested and reviewed before moving to the next increment. This approach ensures that the system is always in a deployable state.

10.2 Completed Increments

1. Authentication and Group Workflows:

- User registration, login, and JWT-based authentication.
- Group creation and member management.

2. Expense and Split Workflows:

- Expense CRUD operations with equal, exact, and percentage splits.
- Validation of splits to ensure accuracy.

3. Balance, Settlement, and Dashboard Workflows:

- Dynamic balance calculation and debt simplification.
- Settlement recording and activity logs.
- Dashboard summaries for cross-group balances.

10.3 Team Contributions

Member	Tasks Completed	Contribution
Shreeya Srivastava	Group and membership integration, group UI, tests, documentation review	25%
Shraddha Sharma	Authentication API, JWT integration, registration/login UI, tests, README review	25%
Rudransh Pratap Singh	Expense APIs, split validation, expense/activity UI, tests, test-case review	25%
Sachit Jain	Balance/debt/settlement APIs, dashboard/balance UI, Docker verification, demo preparation	25%

10.4 Risks and Mitigation

Risk	Mitigation Strategy
Incorrect balance calculations	Automated tests ensure accuracy; balances are dynamically computed.
Unauthorized access	Role-based access control and JWT authentication are enforced.
Deployment issues	Dockerized setup ensures consistency across environments.

11. Challenges Faced and Solutions Attempted

1. Deterministic Rounding:

- Challenge: Ensuring that leftover paise are distributed fairly in equal splits.
- Solution: Allocated leftover paise in ascending user ID order.

2. Debt Simplification:

- Challenge: Reducing the number of transactions required to settle debts.
- Solution: Implemented a greedy algorithm that matches the largest creditor and debtor.

3. Authorization:

- Challenge: Preventing unauthorized access to group data.
- Solution: Enforced membership and admin checks for all sensitive operations.

4. Balance Consistency:

- Challenge: Avoiding stale or incorrect balance data.
 - Solution: Calculated balances dynamically from expenses and settlements.
-

12. Remaining Work

- Conduct a final round of usability testing with real users to identify and fix any UX issues.
- Optimize the performance of the frontend and backend for faster load times and API responses.
- Add a feature to export group expense summaries as a PDF for offline sharing.

13. References

1. [Prisma Documentation](#)
 - Used for database schema management and queries.
2. [Express Documentation](#)
 - Used for building the backend REST API.
3. [React Documentation](#)
 - Used for developing the frontend Single Page Application (SPA).
4. Ian Sommerville, *Software Engineering*
 - Referenced for applying software engineering principles like SDLC and requirements validation.
5. Course lecture material
 - Used for understanding life cycle models, requirements traceability, and software testing.

Appendix A. AI-Use Disclosure

See `AI_USAGE_DISCLOSURE.md` . Students must verify and modify AI-generated content as needed and remain responsible for correctness, security, and originality.